

Product Requirements Document (PRD)

Project Name

Forge AI – AI Software Project Architect

1. Overview

Blueprint AI is an AI-powered web application that transforms software ideas into complete software architecture blueprints.

Users describe an application idea in natural language, and Blueprint AI generates a structured software planning document including architecture, features, API design, database schema, deployment strategy, folder structure, development roadmap, security recommendations, scalability suggestions, and optional code generation for selected components.

The goal is **not to generate an entire application**, but to act as an experienced software architect.

2. Problem Statement

Many developers know what they want to build but struggle with:

- Choosing the correct tech stack
- Designing scalable architecture
- Planning database relationships
- Designing REST APIs
- Organizing project folders
- Planning AWS deployment
- Estimating development effort

Current LLM chat interfaces answer these questions individually but do not produce a structured, reusable software blueprint.

Blueprint AI solves this problem by generating a professional architecture document that developers can immediately begin implementing.

3. Objectives

The application should:

- Generate complete software blueprints
 - Follow industry-standard architecture practices
 - Produce structured, predictable outputs
 - Help developers plan projects before writing code
 - Present results visually instead of long AI conversations
-

4. Target Users

- Students
 - Freelance Developers
 - Startup Founders
 - Software Engineers
 - Hackathon Participants
 - Technical Architects
-

5. Functional Requirements

5.1 Landing Page

Display

- Project Name
- Description
- CTA Button

"Start Building"

5.2 Blueprint Generator

Main feature.

Input

Large Text Area

Example placeholder

Describe your application.

Example:

I want to build a food delivery application where users can order food, restaurants can manage menus, and delivery agents can track deliveries.

Configuration Panel

User selects

Architecture Style

Options

- Monolithic
- Microservices
- Serverless

Database

Options

- PostgreSQL
- MongoDB
- MySQL

Backend

Options

- FastAPI
- ExpressJS

Frontend

Options

- React
- NextJS

Cloud Provider

Options

- AWS
- Azure
- GCP

Project Size

Options

- MVP
- Medium
- Enterprise

Generate Blueprint Button

6. AI Output Structure

The AI MUST return JSON.

Never return plain paragraphs.

Example

```
{  
  "overview": "...",  
  
  "features": {
```

```
"user": [],  
"admin": [],  
"system": []  
},  
  
"techStack": {},  
  
"database": {},  
  
"apis": [],  
  
"folderStructure": {},  
  
"awsArchitecture": {},  
  
"dockerArchitecture": {},  
  
"timeline": [],  
  
"security": [],  
  
"scalability": [],  
  
"futureEnhancements": []  
}
```

Backend converts JSON into UI.

7. Generated Sections

Project Overview

Short description

Features

Separate

User Features

Admin Features

System Features

Recommended Tech Stack

Include

Frontend

Backend

Database

Authentication

Storage

Deployment

CI/CD

Testing

Database Schema

Tables

Relationships

Primary Keys

Foreign Keys

Indexes

API Design

Each endpoint contains

Method

Route

Description

Authentication Required

Sample Request

Sample Response

Folder Structure

Tree View

backend/

frontend/

app/

models/

controllers/

services/

routes/

utils/

config/

Dockerfile

README.md

AWS Architecture

Include

- Frontend Hosting
- Backend Hosting
- Database
- Storage
- Authentication
- CDN
- Load Balancer

Display as a visual flowchart.

Docker Architecture

Show

Frontend Container

↓

Backend Container

↓

Database

Development Roadmap

Phase 1

Planning

Phase 2

Authentication

Phase 3

Core Features

Phase 4

Deployment

Estimated Timeline

Display

Phase	Days
Planning	2
Backend	5
Frontend	5
Testing	3

Security Checklist

- Example
- JWT Authentication
 - HTTPS
 - Password Hashing
 - Input Validation
 - CORS
 - Environment Variables

Scalability Suggestions

Redis

Docker

CDN

Caching

Load Balancer

Microservices

Message Queue

Future Enhancements

AI Recommendations

Analytics

Monitoring

Notifications

CI/CD

8. Interactive Generators

Each section contains Generate buttons.

Example

Database Section

Buttons

Generate SQL

Generate Prisma Schema

Generate Mongo Collections

API Section

Buttons

Generate FastAPI Routes

Generate Express Routes

Generate Swagger Docs

Folder Structure

Generate Folder Tree

Generate README

AWS

Generate Terraform

Generate CloudFormation

Docker

Generate Dockerfile

Generate Docker Compose

9. Export Features

Allow exporting

Markdown

PDF

JSON

10. Save Projects

Store

Project Name

Prompt

Generated Blueprint

Date Created

Last Modified

11. Dashboard

Recent Projects

Favorite Projects

Recently Generated

12. Authentication

Optional.

For MVP

Anonymous usage.

Future

Google Login

GitHub Login

13. UI Design

Style

Modern

Minimal

Dark Theme

Glassmorphism

Inspired by

- Vercel
- Linear
- Cursor
- Claude
- Notion

Animations

Framer Motion

Cards

Rounded

Large spacing

Professional appearance

14. Pages

/

Landing

/generator

/dashboard

/project/:id

/settings

/about

15. Tech Stack

Frontend

React

Vite

TypeScript

TailwindCSS

Framer Motion

React Router

Axios

React Markdown

Lucide Icons

Mermaid.js

React Flow

Shadcn UI

Backend

FastAPI

Python

Pydantic

Uvicorn

OpenAI SDK

SQLAlchemy (optional)

AI

OpenAI GPT-4.1 / GPT-4o

Streaming Responses

Structured JSON Output

Prompt Engineering

Database

SQLite (MVP)

Future

PostgreSQL

Deployment

Docker

AWS App Runner

GitHub Actions

16. Folder Structure

blueprint-ai/

frontend/

components/

pages/

layouts/

hooks/

services/

types/

styles/

backend/

app/

routers/

services/

core/

prompts/

schemas/

models/

utils/

main.py

Dockerfile

docker-compose.yml

README.md

17. Non-Functional Requirements

- Mobile Responsive
 - Streaming AI Responses
 - Secure API Keys
 - Fast Loading
 - Accessible UI
 - Modular Components
 - Error Handling
 - Type Safety
 - Environment Variable Support
-

18. Prompt Engineering Requirements

The backend should never send raw user input directly to the LLM.

Instead, construct a structured system prompt that instructs the model to behave as a senior software architect.

The AI should:

- Return **only valid JSON**.
 - Never generate implementation code unless explicitly requested.
 - Follow a predefined schema.
 - Recommend industry best practices.
 - Avoid hallucinating unsupported technologies.
 - Explain assumptions when requirements are ambiguous.
-

19. Future Scope

- Multi-LLM support (OpenAI, Anthropic, Gemini)
 - Architecture diagram generation using Mermaid
 - ER Diagram visualization
 - Drag-and-drop architecture editor
 - Team collaboration
 - Version history
 - Cost estimation calculator
 - AI-powered risk analysis
 - CI/CD pipeline generation
 - Kubernetes deployment plans
 - Integration with GitHub repositories
-

20. Success Criteria

The application is considered complete when a user can:

1. Describe a software idea.
2. Configure architecture preferences.
3. Generate a structured software blueprint.

4. View the blueprint in organized UI sections.
5. Generate optional artifacts (SQL, API routes, Dockerfile, README).
6. Export the blueprint.
7. Save and revisit previous blueprints.
8. Access the application through a publicly deployed AWS URL.